

Firebird vs PostgreSQL: ACID Transactions and Performance Comparison

Overview of the Two Systems

FirebirdSQL and **PostgreSQL** are both open-source relational databases that fully support ACID transactions. However, they implement transactions and concurrency in different ways, which affects their performance characteristics and data integrity behaviors. Firebird originated from Borland's InterBase and uses a **multi-generational architecture (MGA)** for concurrency, whereas PostgreSQL uses traditional **multi-version concurrency control (MVCC)**. Both approaches provide ACID compliance but differ in **how they manage transaction logs, record versions, and recovery**, leading to distinct performance profiles under various workloads. Below we delve into the technical differences in ACID properties, transaction handling (WAL vs MGA), and how each performs in real-world OLTP and OLAP scenarios.

ACID Compliance in Firebird and PostgreSQL

Both databases are designed to meet **ACID principles** – Atomicity, Consistency, Isolation, and Durability – but they achieve some of these in different ways:

- **Atomicity & Consistency:** Both Firebird and PostgreSQL ensure that a transaction's changes are all-or-nothing (atomic) and that database constraints (e.g. foreign keys, checks) preserve consistency. In Firebird, absolutely *all* operations occur inside a transaction – even single statements – guaranteeing atomic behavior ¹ ². Both systems support features like foreign keys and triggers to maintain consistency (no partial updates that violate constraints) ³ ⁴. They also both support two-phase commit for atomic multi-database transactions when needed ⁵. In practice, there is little difference in how atomicity/consistency are handled – both will rollback an entire transaction on failure and prevent inconsistent state.
- **Isolation (Concurrency Control):** This is where the implementations diverge. **PostgreSQL** supports all four ANSI isolation levels – Read Uncommitted, Read Committed, Repeatable Read, and Serializable ⁶. Its default is **Read Committed**, but it also offers a robust **Serializable** mode for highest isolation. **Firebird**, on the other hand, does *not* implement every ANSI level. It primarily offers **Read Committed** (with two variants) and **Serializable** (often analogous to repeatable-read semantics in Firebird's terms) ⁷. Firebird's two **Read Committed** variants are “read committed *with record version*” versus “read committed *no record version*.” The former is a *snapshot* style read (each query sees the last committed version of each record without waiting), whereas the latter may wait on concurrent uncommitted updates to get the latest data. In effect, Firebird doesn't use a dirty-read mode (no Read Uncommitted) and its “Repeatable Read” is achieved via its MGA snapshot mechanism rather than a separate setting ⁸.

Both Firebird and PostgreSQL use non-blocking multi-version concurrency so that readers do not block writers and vice versa. PostgreSQL's MVCC means each transaction sees a database snapshot as of its start (for Repeatable Read/Serializable) or as of each statement (for Read Committed), avoiding any dirty reads ⁹. Firebird's **MGA** is effectively the same concept: when a transaction updates data, the old version is preserved so that other transactions can continue seeing a consistent older state ¹⁰. A key difference is in configuration and naming: PostgreSQL applies MVCC universally for all isolation levels ¹¹, whereas Firebird always operates in an MVCC mode under the hood but lets you choose if a Read Committed transaction should read the latest committed version immediately or wait for a lock release ¹². In practice, both systems guarantee that one transaction will not see inconsistent partial changes from another – **no dirty reads**, and no lost updates due to the MVCC/MGA design. Both also prevent non-repeatable reads in their snapshot modes. Firebird's **Serializable** mode (sometimes called "Table Stability") can even lock tables for the transaction duration for extra isolation, though this is rarely used. PostgreSQL's highest level, Serializable, uses an MVCC + SSI (Serializable Snapshot Isolation) approach to prevent anomalies.

- **Durability:** Durability ensures committed transactions persist even if the system crashes. Here the approaches differ markedly:
- **PostgreSQL** uses a **Write-Ahead Log (WAL)** to guarantee durability ¹³. Every change is recorded in the WAL on disk *before* the database pages are updated. In the event of a crash, PostgreSQL will replay the WAL upon restart to restore the last committed state. This mechanism, coupled with an automatic recovery routine, ensures no committed transaction is lost (and any partially written changes are rolled back using the log). The trade-off is that after a crash, **recovery involves reading the log and applying changes**, which can mean a longer restart time for very large logs ¹⁴. However, WAL allows powerful features like **point-in-time recovery (PITR)** by archiving log segments – an administrator can restore a backup and replay WAL to a specific point in time, which is extremely useful for disaster recovery ¹⁵.
- **Firebird does not use a WAL or any transaction log for crash recovery** ¹⁶. Instead, Firebird's multi-generational architecture itself ensures durability. When a transaction commits, Firebird writes the data changes directly to the database on disk (typically with *forced-writes* enabled so that the OS cache is flushed on commit). It keeps old record versions around until it knows the changes are durable and no active transaction needs the old version ¹⁷. If a crash occurs, Firebird doesn't need to replay a log; on restart, any transactions that were incomplete simply have their uncommitted new versions discarded, and the previous committed versions of each record remain in place ¹⁸. In essence, Firebird always maintains a consistent state on disk by *never overwriting data in place* without retaining an older copy ¹⁷. This means **startup after a crash is very quick** – there's no lengthy recovery procedure ¹⁹ – and the database is simply ready for use, already in a transactionally consistent state. The downside is that Firebird cannot easily support PITR or incremental restore of changes since it doesn't have an ordered log of changes to apply. Backups in Firebird are typically done via the `gbak` tool (or the newer `nbackup`), and **only full or incremental backups can be taken, without the option to replay a log for point-in-time recovery** ²⁰ ²¹. In summary, PostgreSQL's WAL-based durability provides more **advanced recovery options (e.g. PITR, streaming replication)** at the cost of extra I/O for logging, whereas Firebird's log-free design provides **simplicity and fast crash recovery** at the cost of fewer fine-grained recovery tools. Both databases, when properly configured, will not lose data on power failure – PostgreSQL because of WAL+fsync, and Firebird because of forced-writes (fsync) and preserving prior record versions.

Transaction Implementation: MVCC vs. MGA

Under the hood, both databases manage concurrent transactions with **record versioning**, but the implementation details differ and impact maintenance tasks:

- **Multi-Version Data Storage:** PostgreSQL and Firebird both store multiple versions of a row to avoid writers blocking readers. In PostgreSQL, whenever a row is updated or deleted, a new version (tuple) is written and the old version is left in the table as “dead” (but available to transactions that started earlier) ¹⁰. Firebird’s MGA works the same way: updating or deleting creates a new record version, and the prior version is kept for older transactions ¹⁰. Thus, both fulfill the “**I**” (**I**solation) aspect by providing readers with a consistent snapshot of data without lock contention. One difference is that **Firebird retains old versions until all transactions that might need them finish** ¹⁷ – these versions are not tied to a separate undo log, they live in the main database pages alongside data. PostgreSQL similarly keeps old tuples until it can ascertain no active transaction needs them, but it also uses visibility information (transaction IDs and timestamps) to decide when tuples can be removed.
- **Garbage Collection of Old Versions:** Because both systems accumulate old record versions, they must eventually remove those to reclaim space and maintain performance. **PostgreSQL** relies on **autovacuum** (a background daemon) to periodically scan tables and remove tuples that are no longer needed (committed dead tuples) ²² ²³. Autovacuum kicks in when a table has a significant number of updates or deletes – it uses thresholds to decide when to vacuum a table that had a “large number of inserted, updated or deleted tuples” ²² ²³. If autovacuum is off or behind, an explicit `VACUUM` can be run. Until vacuum runs, PostgreSQL’s dead tuples remain in the table (they are invisible to other transactions, but still occupy space). Queries will skip over them, especially if using index-only scans that leverage the visibility map to avoid reading pages known to have no live tuples ²⁴ ²⁵. This means that after a massive update/delete, PostgreSQL might experience bloat (the table file becomes larger with dead rows) and potentially slower sequential scans (scanning many dead rows) until vacuum cleans up. However, the *cleanup cost is paid by the vacuum process* asynchronously, not by the user query. In other words, PostgreSQL opts to defer cleanup to background maintenance to keep foreground query latency predictable.

Firebird handles garbage collection differently. Firebird can perform garbage collection *cooperatively* or in the *background*, depending on configuration and server model ²⁶ ²⁷. In **cooperative GC**, when a user’s query encounters old record versions (garbage), that same query process will also clean them up. This “on-the-fly” cleanup can cause a noticeable slowdown if a query triggers cleanup of a large amount of garbage. For example, if you delete millions of records in Firebird and then immediately perform a full table scan (`SELECT *` or similar), that select operation will not only read data but also spend time **rewriting pages to purge the deleted record versions** it finds, **making the query much slower** ²⁸. The benefit is that garbage is removed sooner and continuously by the workload. Firebird’s **background GC** (available in the Superserver architecture) instead offloads that work to a separate thread: when a transaction commits that created garbage, or when a query sees garbage, it signals a background thread to wake up and clean those record versions in the background ²⁷. By default in modern Firebird (Superserver), a hybrid approach is used – some cooperative cleanup with a dedicated GC thread also running ²⁶. The **impact on performance** is that Firebird might pause user transactions for cleanup in cooperative mode, whereas in background mode garbage might linger a bit longer (slightly more storage use) but user transactions remain fast ²⁹. Administrators can tune this via the `GCPolicy` setting (`cooperative`, `background`, or `combined`) ²⁶ ²⁹. Additionally, Firebird provides a manual **sweep** (`gfix -sweep`) and the backup

tool (`gbak`) also performs garbage collection by default to fully clear old versions ³⁰. In summary, **PostgreSQL's vacuum and Firebird's GC/sweep serve the same purpose** – both require periodic cleanup of old row versions. PostgreSQL's approach keeps cleanup entirely in the background (unless autovacuum is misconfigured), whereas Firebird might impose some cleanup cost on transactions unless configured purely for background GC. If neither cleanup mechanism is run, **both databases will suffer eventual slowdowns or excessive disk usage**: PostgreSQL from table bloat and transaction ID wraparound risk, and Firebird from long chains of old record versions that make reads slower ³¹.

- **Transaction ID Handling:** Both systems use transaction IDs to track visibility of versions. PostgreSQL has a 32-bit transaction ID that can wrap around, which is why vacuum must also “freeze” old rows after ~2 billion transactions to prevent confusion ³² ³³. Firebird also has a Transaction Inventory and Oldest Active Transaction tracking; if long-lived transactions prevent cleanup, the gap between oldest and newest transactions can grow and eventually force a sweep to avoid ID wraparound issues. In practice, both require avoiding extremely long-running transactions for good health. A long-running snapshot transaction in Firebird can retain very old record versions and prevent their removal just like a long-running transaction in PostgreSQL can prevent vacuum from removing dead tuples. Best practice in both systems is to **commit transactions promptly** in OLTP systems to let cleanup keep up.

Performance Under Load: OLTP and OLAP

OLTP Performance and Concurrency

For **online transactional processing (OLTP)** – many short transactions, concurrent updates/inserts, and high throughput – both Firebird and PostgreSQL leverage MVCC to maintain high concurrency. **Readers never block writers** in either system, which is a major advantage over purely locking engines. This design yields excellent scalability for concurrent users on both platforms.

Key points for OLTP and concurrency:

- **Connection Architecture:** PostgreSQL uses a multi-process model (each connection is a process, with shared buffer memory and coordination by locks/latches), while Firebird offers multiple models. Firebird 3+ Superserver is multi-threaded in one process for all connections (classic mode spawns separate processes per connection, more like Postgres). In practice, modern Firebird Superserver can handle many simultaneous users efficiently, as it has improved threading and lock management over older versions. There is no hard-coded connection limit in either (both can handle hundreds of connections, though very high connection counts might need pooling in Postgres due to memory overhead per process).
- **Throughput and Transaction Rate:** Both databases can handle high transaction rates (hundreds of commits per second) on adequate hardware. Firebird is often praised for being lightweight with **fast context switching between transactions** and a small footprint, which can benefit OLTP throughput on smaller machines. In user reports, Firebird demonstrates *“exceptional speed and stability, particularly in scenarios with numerous concurrent connections,”* often making it a top choice for embedded systems or small-to-mid size business applications ³⁴ ³⁵. PostgreSQL is known to scale well on larger multi-core servers and can maintain high throughput even as data size grows, albeit

with careful tuning of parameters like checkpoint frequency, background writer, and autovacuum to sustain performance.

- **Locking Behavior:** Thanks to MVCC, **reads do not lock writes** in either system. Writes in both will acquire row-level locks on the modified records (so two transactions editing the same row will conflict in both Firebird and Postgres). Both databases avoid escalating to table-level locks for data modifications (except certain DDL operations). A notable difference: **Firebird can place an update conflict immediately if two transactions modify the same row, whereas PostgreSQL will wait** (in Read Committed) for the first transaction to commit, then abort the second with a serialization error if using Serializable or let it proceed if using Read Committed and the first committed (the second would then see the updated data as a conflict on update). In Firebird, the second updater will get an update conflict error that must be handled (this is similar to optimistic concurrency patterns).
- **Deadlocks and Conflict Resolution:** Both systems detect deadlocks (cycles of waiting transactions) and abort one of the transactions to resolve it. PostgreSQL's lock manager will detect deadlock between any locks (row locks, etc.) and report an error. Firebird's engine will report update conflicts or deadlocks similarly. There is no fundamental performance difference here; both handle deadlocks automatically.
- **Scaling with Cores:** PostgreSQL can utilize multiple CPU cores *across* separate connections easily (each backend process runs on a core as needed). A single complex query in PostgreSQL can also be parallelized (since version 9.6/10, PostgreSQL has parallel query for scans, joins, aggregations in OLAP workloads). Firebird historically executed each query single-threaded, but as of Firebird 5, there are improvements like parallel index builds and parallel sort operations. Still, **Firebird's query execution is largely one-thread-per-query** (the Firebird 5 release introduced some parallel execution for certain maintenance and scan tasks ³⁶). In high-concurrency OLTP (many short queries), both databases leverage multiple cores effectively by handling different connections on different threads/processes. Firebird 4 and 5's improvements mean it can utilize multi-core hardware much better than older Firebird 2.x could. PostgreSQL's overhead per connection (process) is higher, which is often mitigated with connection pooling in high-concurrency scenarios, whereas Firebird's lightweight thread approach can sometimes handle lots of simultaneous connections without an external pool.

In summary, **for OLTP with moderate-sized data**, both databases perform very well. Firebird is often chosen for its **ease of use and low administration overhead** for smaller systems, and it *"is known for its user-friendliness"* and minimal setup, yet solid performance ³⁷. PostgreSQL is known for **robustness at high volume**; it has been proven in very large-scale systems and can exploit advanced hardware and tuning. Both ensure data integrity under high load – you won't lose committed data in either system even under power failures or crashes (given proper configuration). PostgreSQL's WAL offers extra safety in that you can stream the log to replicas for high availability; Firebird starting with version 4 introduced its own replication feature as well, but that's outside ACID scope.

Handling Large Transactions (Mass Updates/Deletes)

A particular point of comparison is how each database copes with *very large* transactions – for example, an `UPDATE` or `DELETE` affecting millions of rows in one go (a batch job or maintenance task). Traditional engines like SQL Server often recommend breaking such operations into chunks to avoid huge locks or

transaction log growth. With Firebird and PostgreSQL, the MVCC design means that **large write transactions are possible without blocking reads**, but there are performance considerations:

- **Firebird:** A huge update or delete will create a large number of new record versions or deletion stubs. Firebird can handle this in one transaction (there is no built-in limit on transaction size aside from resources). All those changes will reside in memory (transaction state) and on disk in the database pages until commit. **During the large transaction, readers are not blocked** – they will continue to see old versions of records. Once the transaction commits, all those old versions become garbage to be collected. The immediate impact is that the commit itself might be slow if forced writes are on (flushing all changed pages to disk). After commit, if a lot of garbage was generated, and if the system is using cooperative cleanup, the *next reader that touches those pages may incur a big cleanup cost*. For example, deleting 10 million rows in Firebird and then selecting from that table will likely trigger garbage collection of those 10 million old versions, which can make the select temporarily “unresponsive” as it cleans up ²⁸. There are strategies to mitigate this: one is to enable background GC (so that the garbage is cleaned asynchronously by Firebird’s thread, not by the user query) ²⁷. Another strategy is to run a manual sweep or a dummy `SELECT COUNT(*)` after the big delete in a controlled way to force garbage collection before your real workload continues ³⁸. Firebird’s documentation often suggests that if you must delete or update huge numbers of rows, you might consider doing it in manageable chunks or during off-peak hours, not because of engine failure risk, but to reduce the impact of garbage accumulation on the active users. Notably, Firebird *does not have a fast “TRUNCATE” command* for tables – deleting all rows means each row version is handled individually. If you need to wipe a table, backing it up and restoring or recreating it empty can be faster.
- **PostgreSQL:** A large delete/update in PostgreSQL will also succeed in a single transaction, and it likewise will not block other transactions from reading other data (they’ll see old versions as well). PostgreSQL will, however, write a vast amount of WAL (every deleted or new tuple is logged). This can impact performance: the I/O of writing and later flushing the WAL might be the slowest part of the operation. The commit in PostgreSQL will flush the WAL records to disk (ensuring durability). Following the commit, the table in question will be bloated with dead tuples. **Unlike Firebird, PostgreSQL will not clean them immediately on commit** – it relies on autovacuum to reclaim that space later ³⁹ ⁴⁰. This means that right after a massive delete, a sequential scan on that table will read through a lot of dead rows (which it can identify as dead, so correctness isn’t an issue, but it’s wasted I/O). Autovacuum usually detects the “large number of deleted tuples” and will start vacuuming the table in the background relatively soon ⁴¹ ³⁹ (unless autovacuum is turned off or the thresholds are set too high). The vacuum process will run in parallel to user queries, gradually cleaning pages. During vacuum, PostgreSQL *does* require a maintenance lock that can coexist with normal reads/writes, so typically it doesn’t block normal operations (except it may pause if it encounters a conflicting lock). Thus, after a big data purge, Postgres will eventually compact the table and free the space (or mark it for reuse) without developer intervention, but **the storage bloat and I/O cost exist until that happens**. As a result, DBAs sometimes choose to manually vacuum or even `CLUSTER` or `REINDEX` after huge data changes to optimize performance. Also, PostgreSQL offers a **TRUNCATE** command which instantly removes all rows in a table (much faster than a DELETE of each row, but it requires an exclusive lock on the table). If the goal is to delete everything, TRUNCATE is far more efficient in Postgres.

Summary – large operations: Both systems can perform large batch modifications in a single transaction without breaking integrity. Neither will lock the entire table in a way that stops concurrent reads (except that PostgreSQL's TRUNCATE does take an exclusive lock, and Firebird's lack of TRUNCATE means you have to DELETE which is MVCC-friendly). The performance impact is mainly on **cleanup and I/O**: PostgreSQL will generate a lot of WAL and then need to vacuum; Firebird will generate a lot of garbage versions and may need to sweep/GC. In practice, many users of both databases **still opt to do large deletions in smaller chunks** (e.g. delete 100k rows at a time commit, then next 100k) if doing it online, to avoid long stalls – this is more for operational convenience than a hard requirement. Unlike SQL Server, you won't hit a transaction log capacity issue as long as disk can handle it, but you might find the transaction running for a long time. A long-running huge transaction can also keep older versions of *other* tables from being cleaned in both systems (because the transaction is not finished). So chunking can help the system keep up with vacuum/GC in parallel.

OLAP and Large Dataset Performance

For **analytical queries or large data sets (OLAP)**, the differences between Firebird and PostgreSQL become more pronounced. PostgreSQL has, over the years, added many features aimed at complex queries on large datasets, whereas Firebird has remained a leaner system focused primarily on OLTP-style usage. Here's how they compare:

- **Query Optimizer and Complex Joins:** PostgreSQL's query planner is very sophisticated, handling multi-table joins, subqueries, window functions, CTEs, etc., and it can use advanced techniques like **parallel query execution**, custom indexing (GIN/GIST for full-text, SP-Gist, BRIN for large data), partition pruning, and more. Firebird's optimizer is capable for typical queries, but it historically did not have intra-query parallelism or some of the advanced index types. Firebird 4 introduced basic window function support and Firebird 5 improved the optimizer and added **partial index** capability and parallel index builds ⁴², but it **still does not parallelize a single SQL query across multiple CPU cores** in the general case. This means for very large aggregations or data warehouse-style queries, PostgreSQL can utilize multiple processors and more advanced algorithms, often resulting in faster execution on big data.
- **Large Database Scalability:** Users have reported that Firebird works very well up to databases of a few gigabytes or even tens of GB, but **performance can degrade with very large datasets** (hundreds of GB or more) due to memory usage and the absence of features like partitioning. In fact, Firebird is often cited as *struggling to handle very large databases compared to enterprise-grade systems* – “while Firebird works well for small to medium-sized databases, it may struggle to scale and handle very large datasets” ⁴³. One reason is that Firebird keeps a relatively simple storage engine without partitioning or tablespaces; a single table with 100 million rows might be harder to manage in Firebird (e.g. index maintenance, backup time) than in PostgreSQL which offers table partitioning and sophisticated indexing. PostgreSQL, on the other hand, “*excels in handling large datasets and high-traffic applications*” with features like table partitioning, replication for scale-out reads, and connection poolers for scale-up ⁴⁴. It's not unusual to find Postgres databases in the terabyte range powering analytics or heavy batch reporting.
- **OLAP Workloads:** For intensive read-only analytical workloads (scanning lots of data, grouping, etc.), PostgreSQL tends to outperform Firebird primarily because of its ability to use parallel workers and more efficient query execution for these patterns. Firebird's sweet spot is embedded or smaller-scale

transactional workloads. It lacks built-in parallel query, and as a result a full table scan on a huge Firebird table is single-threaded. Moreover, Firebird's MGA means that long-running read transactions could accumulate a lot of garbage if writes are happening concurrently, which might not be ideal for reporting queries running concurrently with updates (PostgreSQL has to deal with a similar issue, but autovacuum and HOT updates mitigate some effects).

- **Use in Data Warehousing:** PostgreSQL is often used as a backend for data warehouses and can be tuned for bulk loading and large aggregations. It also has extensions like Foreign Data Wrappers to pull in external big data, and FDW-based partitioning across servers. Firebird is rarely used in data warehouse scenarios – its design is more oriented to operational databases. Firebird has fewer tools for big-data manipulation and lacks things like grouping sets, JSON aggregation (Postgres has JSON/JSONB with indexing), etc., which are useful in modern analytic applications.
- **Memory and Disk I/O:** Firebird's footprint is small and it can run efficiently on modest hardware, but for big analytics, you often want to cache large portions of data in memory. PostgreSQL allows configuration of large shared buffers and `work_mem` for sorting/hashing operations. Firebird also allows some tuning of cache buffers per database. However, because Postgres can use OS cache plus its shared buffers, and because it can do index-only scans (thanks to visibility map) to reduce I/O on large tables, it often has an edge in heavy read scenarios. Firebird's index implementations and binary sort performance are good for OLTP (small range scans), but for large range scans or full scans Postgres may be more I/O-efficient due to decades of refinements for large systems.

Bottom line for OLAP: If your use case involves heavy analytical querying on large data (especially if data size grows into dozens of GB or more), PostgreSQL is generally the better fit due to its feature set and optimizations for such scenarios. Firebird can certainly run reports and has decent read performance on reasonably sized data sets, but it *“may decline in efficiency when dealing with massive datasets”* ³⁵. On the flip side, Firebird's simplicity can shine in small-to-medium OLTP and mixed workloads – it's lightweight and often requires less tweaking to get good performance out of the box for common workloads.

High-Level Summary of Differences (Firebird 5 vs PostgreSQL 15/16)

For those less familiar with the technical details, here is a brief summary of how Firebird and PostgreSQL differ in transaction handling and what it means for performance and integrity:

- **Concurrency Model:** Both Firebird and PostgreSQL use multi-versioning to allow many users to read/write at the same time without blocking. Firebird calls its approach **MGA (Multi-Generational Architecture)**, and PostgreSQL calls it **MVCC (Multi-Version Concurrency Control)**. In practice, this means neither will lock a table for reading while it's being written – they keep old versions of data so reads are consistent ¹⁷ ⁴⁵. This design gives **high concurrency and avoids most read/write lock contention** in both systems.
- **Transaction Isolation Levels:** PostgreSQL supports the full range of standard SQL isolation levels (including the advanced Serializable mode), giving developers flexibility to choose how strict the isolation should be ⁶. Firebird has equivalent functionality but with fewer modes (it primarily uses snapshot isolation and a read-committed mode with optional record versioning) ⁷. Practically, Firebird's default “snapshot” isolation is similar to PostgreSQL's Repeatable Read – both prevent dirty

or non-repeatable reads by using versioned data. Firebird doesn't allow a "dirty read" (Read Uncommitted) at all, which is usually fine since that level is rarely used.

- **Durability and Logging:** PostgreSQL uses a **Write-Ahead Log (WAL)** to record every transaction's changes on disk before they're applied to the main database files ¹³. Firebird **does not use a WAL** – it writes changes directly to the database and keeps older versions around as a safety net ¹⁷. Both approaches ensure that if the database crashes, you won't lose committed data. PostgreSQL might take a bit longer to start after a crash because it **replays the log to reconcile the database state** ¹⁴. Firebird, after a crash, just discards any uncommitted changes and is ready to go, since the last committed state was already on disk (no lengthy recovery needed) ¹⁸ ⁴⁶. PostgreSQL's WAL enables advanced features like point-in-time recovery and streaming replication for high availability ¹⁵. Firebird relies on backups (and as of v4, a replication feature) for recovery but cannot restore to an arbitrary point in time without a snapshot backup from that moment ²¹.
- **Maintenance (Vacuum vs Sweep/Garbage Collection):** Both databases require some background work to clean up old row versions that accumulate due to MVCC/MGA. PostgreSQL automatically **vacuums** tables in the background – this frees space from deleted or outdated rows and prevents transaction ID wraparound issues ³⁹ ⁴⁰. Firebird automatically performs **garbage collection** of old record versions, either during normal queries (cooperative) or via a background thread (Superserver) ²⁶ ²⁷. In PostgreSQL, you might occasionally notice increased I/O due to autovacuum, but it's usually lazy and avoids impacting query performance significantly. In Firebird, if not tuned, you might notice a query pausing to clean up a lot of "garbage" after heavy updates/deletes ²⁸. DBAs can configure Firebird to do that cleanup in the background to reduce impact ²⁹. Regular backups and the sweep utility in Firebird serve a similar purpose to aggressive vacuuming in PostgreSQL – keeping the database neat and performant.
- **Large Batch Operations:** If you delete or update millions of rows, neither database will corrupt or choke, but the aftermath differs. In PostgreSQL, the database file might temporarily bloat (those rows turned into dead tuples) until autovacuum cleans them; queries won't stall, but you're carrying some baggage until cleanup happens. In Firebird, those millions of deletions create millions of old versions that *might* be cleaned during the next query or by a background thread – if it happens during your query, you'll feel a slowdown as it cleans up ²⁸. You can configure or manage this (by scheduling a separate sweep or enabling background GC) to avoid impacting users. Essentially, **both can do large transactions safely**, but some planning (like doing such operations in off-hours or in chunks) is wise so that the cleanup work doesn't surprise you.
- **Performance and Scalability:** Firebird is a **small, efficient engine** that excels with many simultaneous users on moderate data sizes. It's known for *fast performance on read/write workloads with tens or hundreds of concurrent connections*, and it has a small footprint ³⁴. However, as data grows very large (hundreds of millions of records), Firebird can become slower or require more careful tuning – users note that *Firebird's efficiency may decline with very large databases* ³⁵. PostgreSQL is **heavier** in terms of resource usage, but it scales up to much larger systems – it's routinely used for large-scale applications and data warehouses. PostgreSQL can handle large datasets (multi-GB to TB range) more comfortably, and it has features like partitioning, parallel query execution, and numerous indexing options that help with big data and complex queries. For pure OLTP (lots of small transactions), both are very capable; for mixed workloads or OLAP queries, PostgreSQL's advanced planner and parallelism give it an edge in throughput.

Both Firebird and PostgreSQL are **robust, ACID-compliant databases**. They ensure data integrity through slightly different means – Firebird by a careful no-overwrite architecture (no separate log file), and PostgreSQL by a write-ahead logging design. In practice, either can be used for mission-critical systems. The choice often comes down to the specific use case: if you need an embeddable, zero-admin, lightweight DB with solid transaction support, Firebird is attractive. If you need enterprise features, scalability to huge workloads, and a broad ecosystem of tools, PostgreSQL is often the better choice. Each has strengths: Firebird's **simplicity and concurrency** vs. PostgreSQL's **feature depth and proven scalability**.

1 2 3 4 5 13 14 16 17 18 19 46 ACID Compliance and Firebird

https://www.firebirdsql.org/file/documentation/papers_presentations/html/paper-fbent-acid.html

6 7 8 9 11 12 15 20 21 45 Differences between PostgreSQL and Firebird in Transaction Management - VPS Hosting, Webhosting a Dedikované Servery

<https://www.mydreams.cz/en/hosting-wiki/6205-differences-between-postgresql-and-firebird-in-transaction-management.html>

10 26 27 28 29 30 31 38 How to delete large data from Firebird SQL database - Stack Overflow

<https://stackoverflow.com/questions/66265900/how-to-delete-large-data-from-firebird-sql-database>

22 23 24 25 32 33 39 40 41 PostgreSQL: Documentation: 18: 24.1. Routine Vacuuming

<https://www.postgresql.org/docs/current/routine-vacuuming.html>

34 35 37 43 44 PostgreSQL vs Firebird | Which Relational Databases Wins In 2026?

<https://www.selecthub.com/relational-database-solutions/postgresql-vs-firebird-database/>

36 Detailed New Features of Firebird 5, Part 4: Parallel Features

<https://ib-aid.com/br/articles/detailed-new-features-of-firebird-5-part-4-parallel-features-backup-restore-sweep-creation-of-indices/>

42 Firebird 5.0.0 DBMS Release: Best New Features - DebugPoint.com

<https://www.debugpoint.com/firebird-5-0-0/>